Claude Code for Vibe Coding

Trainer: Goh Kheng Yee

TRAINER PROFILE



Trainer Kheng Yee

- HRDC Accredited Trainer
- M.Sc. in Analytics, Georgia Tech, USA
- 6 years of Industrial Experience in AI & Data Science
- Trainings focused on AI, Machine Learning, Python, SQL, Data Analytics, and Data Visualization.

Professional Experience Highlights

Consulting

Led development of AI-powered learner engagement systems for an education institution. Contributed to public sector projects by applying advanced analytical models.

Media

Built AI/machine learning models to identify audience segments, enabling the shift to addressable TV advertising.

Banking

Developed machine learning models for banking products and optimized lead quality through model overlay techniques.

Professional Certificates

HRDC Accredited Trainer

AWS Certified Cloud Practitioner

Salesforce Certified Tableau Desktop Foundations

Google Cloud Certified Cloud Digital Leader

© 2025 InferLens Training Services | All rights reserved.

Introduction to vibe coding with Claude Code

01

Describe

State the outcome in ordinary language.

02

Generate

Ask Claude to propose and make a small change.

03

Observe

Run the result and compare it with your goal.

04

Refine

Use the evidence to request the next change.

Human review and testing belong in every cycle.

Claude assists. You remain accountable.

CLAUDE CAN HELP

Explain unfamiliar files
Draft code and tests
Suggest a plan or a fix
Refactor repeated logic
Summarise the changes

YOU STILL DECIDE

What the application should do
Which actions to approve
Whether data is appropriate
Whether tests are sufficient
Whether a result is safe to use

Sonnet, Opus, Fable and Haiku

Model	Positioning and trade-off	Example coding use
Sonnet	Fast, lower-cost option for well-scoped everyday work	Build a feature, fix a bug or review a small change
Opus	Strong general starting point; higher cost than Sonnet	Handle a complex feature or a difficult debugging task
Fable	Demanding reasoning and long-running agents; higher cost	Investigate a difficult problem when Opus falls short
Haiku	Fastest small model for high-volume, cost-sensitive work	Summarise files or run a focused research subagent

Inside a session: slash commands

/init

Create a starter CLAUDE.md for the project

/compact

Summarize the chat to free up context

/model

Switch models mid-session

/memory

Edit memory files

/clear

Start fresh between unrelated tasks

/rewind

Roll code and conversation back

/permissions

Choose which tools run without asking

/review

Get a code review of your changes

Write prompts Claude can check itself against

Vibe coding works best when Claude knows what “done” looks like

01	**Give it a test to pass.**	Include example inputs and expected outputs, then ask Claude to run the tests and fix failures.
02	**Be specific.**	Name the file, the scenario, and any constraints (e.g. “no new libraries”).
03	**Point to an existing pattern.**	“Follow how HotDogWidget.php is built” beats describing the style yourself.
04	**Describe symptoms, not just goals.**	Paste the actual error and ask for a root-cause fix.
05	**Plan before big changes.**	Use plan mode (Shift+Tab) to explore and plan first; skip it for one-line fixes.

Source: https://code.claude.com/docs/en/best-practices

The 5-Line Vibe Coding Prompt

Example: Split bill calculator

01	**What?**	Build me a bill-splitting app called Split.
02	**Who?**	It’s for my friends and me after group dinners, when nobody wants to do the math at the table.
03	**What does it do?**	I enter the total, the tip percentage, and how many people, and it shows what each person pays.
04	**What does it look like?**	Clean and bright, big numbers, easy to read on a phone in a dim restaurant.
05	**What does it never do?**	It should never make me sign up or ask for anything beyond the numbers.

Prompt structure: Kopi Corner example

Part	Question it answers	Example from the lab
Goal	What are we making, and for whom?	One-page menu for Kopi Corner
Audience	Who uses it, and where?	Customers on their phone in the queue
Features	What must it do or show?	3 sections, name, description, price in RM
Constraints	What are the limits or rules?	One file, no libraries, mobile-friendly
Output	What should Claude hand back?	The file plus a 3-bullet summary

Manage the session, not just the prompt

Context is the scarcest resource. Protect it.

01	**One task per session.**	Run /clear between unrelated tasks.
02	**Two strikes, restart.**	If you've corrected Claude twice on the same issue, clear and rewrite the prompt with what you learned.
03	**Let Claude interview you.**	For bigger features, ask Claude to question you and write a SPEC.md, then build it in a fresh session.
04	**Delegate research.**	"Use subagents to investigate X" keeps exploration out of your main context.
05	**Keep CLAUDE.md short.**	Only include what Claude would get wrong without it.
06	**Get a second opinion.**	Have a subagent review the diff before you call it done.

Rapid application prototyping

01	**Frontend**	What you see: a form, buttons and a list of tasks.
02	**Backend**	What handles requests: check inputs and apply the task rules.
03	**Database**	What remembers data: save tasks so they survive a restart.

CRUD describes four basic app behaviours

Action	Task tracker example	A check you can observe
Create	Add “Read course notes”	One new task appears
Read	Open the task list	Saved tasks appear
Update	Mark the task complete	Its status changes
Delete	Remove the task	It no longer appears

AI-assisted debugging begins with evidence

Symptom	Evidence to collect	Next question
App will not start	Exact command and full error message	Which step failed?
Page opens, action fails	Action taken and visible error	Which request failed?
Result is wrong	Expected result and actual result	Which rule is missing?
Old feature breaks	Last change and a repeatable example	What changed?

A debugging prompt that supplies evidence

EXAMPLE PROMPT

Include an exact error message when there is one.

Never replace evidence with “it is broken.”

When I add a task with a blank title, the app saves an empty row.

Expected: show “Please enter a title” and save nothing.

Reproduce the issue, explain the likely cause and propose the smallest fix. Preserve the other features. Add a test and report which checks you actually ran.

Workflow automation and scripting

01

Input

A folder, a file or a request.

02

Processing

Rules that transform or summarise the input.

03

Output

A report or a controlled change.

Python handles our file-report lab. Node.js runs our task tracker.

Script-writing prompt template

01	**Task and inputs**	Write a script that [does the task] for [these files / this data].
02	**Rules**	[dry run first / never delete / ask before changing anything].
03	**Output**	[what file or summary I want at the end].

Working with existing codebases

01	**What is here?**	Ask for the main folders, the entry point and a plain-language purpose.
02	**What does it depend on?**	Ask which packages, services and configuration values it needs.
03	**Where should I change it?**	Locate the smallest relevant area and the checks that protect it.

CLAUDE.md records project guidance

```
# Course task tracker

Use fictional data only.
Keep the app local.
Prefer small, readable changes.
Explain commands before running them.
Preserve the existing acceptance checks.
Report tests that you could not run.
```

PROJECT CONTEXT

A shared place for conventions and working instructions.

Keep secrets out.
Keep it up to date.

A useful code review asks for evidence

Priority	Question	Evidence to request
Correctness	Does the change meet the requirement?	A reproducible check
Scope	Did it alter unrelated behaviour?	A changed-file summary
Readability	Can someone understand the change?	Names and concise explanations
Safety	Does it expose data or add risky access?	Data flow and permission review

Building an end-to-end project

PROJECT BRIEF

A local course task tracker that keeps tasks between sessions.

The interface, server logic and database must work together.

REQUIRED BEHAVIOURS

Add a task with a non-empty title
List saved tasks
Mark a task complete
Delete a selected task
Retain data after restart

Project milestones keep the work manageable

01	**Agree the brief**	Confirm features, constraints and acceptance checks.
02	**Plan the structure**	Name the interface, server and storage responsibilities.
03	**Build a thin slice**	Make one task travel from the form to storage and back.
04	**Expand carefully**	Add remaining behaviours one at a time.
05	**Review and demonstrate**	Run checks and disclose unfinished work.

Responsible and effective vibe coding

01	**Incorrect output**	Compare code and behaviour with explicit requirements.
02	**Sensitive information**	Use sample data and approved environments.
03	**Risky actions**	Review commands, permissions and dependency changes.
04	**False confidence**	Record actual tests and ask for human review.

Skills in Claude

A skill is a reusable capability for a specific task

WHAT IT CONTAINS

Name + description

Markdown instructions

Optional scripts, references, templates and other resources

WHY USE SKILLS?

Reusable	Use across agents
Modular	Keep tasks focused
Shareable	Export files or packages
Clear	Understand and maintain behavior

Skills in Claude

project-status-dashboard/

SKILL.md	Required: when to use the skill + main instructions
references/	Knowledge the agent reads
scripts/	Code the agent runs
assets/	Files the agent uses in its output

Claude Code Remote Control

Start at your desk, continue from anywhere

Drive a Claude Code session running on your own machine from claude.ai/code or the Claude mobile app.

RUNS LOCALLY

Code, files, servers and tools stay local. Your phone or browser displays the session.

STAYS IN SYNC

Get push alerts when tasks finish or need input.

EASY TO START

Run claude rc in a session, then scan the QR code.

Requires Pro, Max, Team or Enterprise. Team/Enterprise need admin enablement.
No API-key authentication.